



Softwarearchitektur & Architekturdokumentation

Programm für heute



QA|WARE

Vorlesung

1. Was bedeutet “gute” Softwarearchitektur
2. Architekturentscheidungen mit Architecture Decision Records (ADR)
3. Softwarearchitekturdokumentation mit arc42

Übung

1. Architecture Decision Records (ADR) anwenden
2. Übung zur Dokumentation mit arc42
3. Vorstellung der Ergebnisse in Kleingruppen (ca. 10min)

Was interessiert euch zum Thema Testen?



QA|WARE





QA|WARE

Was bedeutet “gute” Softwarearchitektur

Es gibt vier Architekturebenen.

- *Architektur im ganz Kleinen*: macht der Programmierer (aka Design, Micro-Architecture)
- **Architektur (im Kleinen)**: Architektur eines (Teil-)Systems. Thema von zahllosen Architektur-Büchern (Foundations, Moderne SW-Architektur, ...). Aufgabe von A- und T-Architekten.
- Architektur im Großen: Architektur von mehreren kooperierenden Systemen. Ebenfalls Aufgabe von A- und T-Architekten. Fokus häufig auf “mein” System: wir und die anderen.
- *Architektur im ganz Großen* (aka Enterprise Architecture): Architektur einer Landschaft von vielen hundert Systemen, z.B. Systemlandschaft von BMW. Eigenes Thema, zahllose Ansätze.

Architekturkonzepte



QAWARE

- *Komponenten und Schnittstellen* sind die zentralen Einheiten des Entwurfs. Innensicht und Außensicht.
- *Datentypen* definieren die Sprache einer Komponente: technische/fachliche Datentypen, altes/neues Schlüsselsystem
- *Schichten/Schalen* repräsentieren eine technische Struktur (z.B. DB, AWK, UI, Proxies)
- *Säulen* repräsentieren eine fachliche Struktur (z.B. Verkauf, Lager, Reklamation, Inkasso).
- Wir zerlegen Säulen in Schichten/Schalen → z.B. *Hexagon, Zwiebel, Microservices*
- Architekturstile:
 - *Layered, Service Based, Orchestration Driven, Event Driven, Microservices, Space-Based*
 - *Microkernel*
 - *Pipeline*

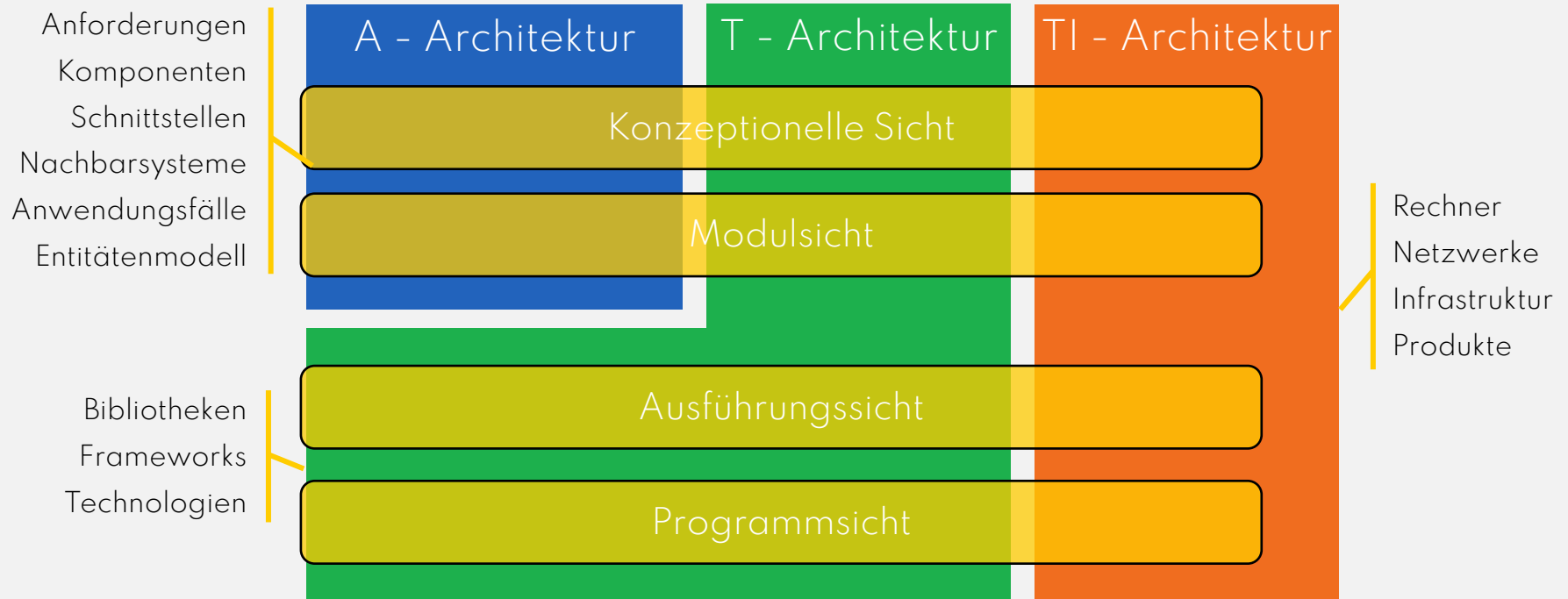
Wie designed man solch eine Struktur.



QA|WARE

- Welche Systeme gibt es, welche sind wichtig (d.h. “wir” kommunizieren mit denen)
- Für jedes System:
 - welche Daten sind da drin?
 - Wo kommen die her? Upstream, Benutzer, ..?
 - Wer braucht die Daten? Downstream, Benutzer, ..?
 - Was ist die Wertschöpfung?
- Male ein Bild, das den Datenfluss/die Wertschöpfungskette anschaulich macht
- Analysiere jede Schnittstelle/Übertragungskanal:
 - Welche Daten, welche Datenmengen
 - Welche Aktualität, welche Qualität
 - Was kann da schiefgehen? (Daten bleiben hängen, kommen zweimal, die falschen Daten kommen)
 - Synchron/Asynchron/Batch?
- Technische Optionen (nur die wichtigsten) -
 - Queue (Kafka?), Call (gRPC, REST, GraphQL), Batch?
 - Datenhaltung: Datenbank (SQL, NoSQL, Ignite, ..); wo ist der Point of Truth; Redundanzfreiheit vs. Schnelligkeit, ...
- Evolution: IST-Stand, Ziel-Architektur wird in mehreren Schritten erreicht; verschiedene Ausroll-Strategien möglich (pro Land, pro Benutzergruppe, pro Datenbereich, z.B. Baureihe), wie soll die Welt in fünf Jahren aussehen?

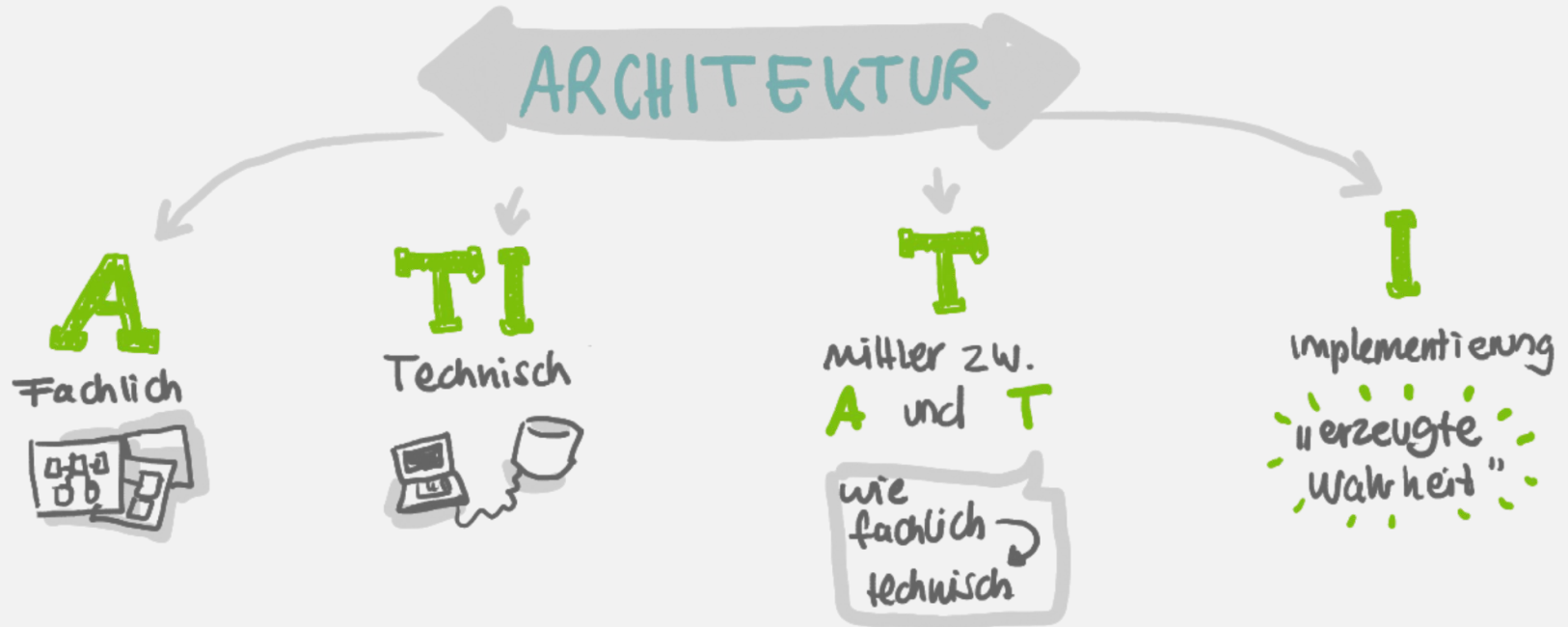
Softwarearchitekturen und Architektursichten.



Die vier Architekturansichten



QA|WARE



A - ARCHITEKTUR



Eigenschaften

Datenhoheit
"Die gehören mir"



Lebensdauer? ⌚



Zuständigkeit
Anlegen
Löschen
Ändern

Problem Hiding

Abhängigkeiten --

Unabhängigkeit ++



Komponenten finden

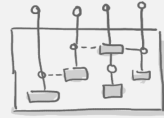
Datenmodell

Use Cases

Anforderungen



A - ARCHITEKTUR



Ist meine
A-Architektur
Simvoll??

a)

gucke **Anforderungen**

↓
wie wird ändern?

hält system dem
stand?



new

↳ was passiert?

b)

Schreiblischtests

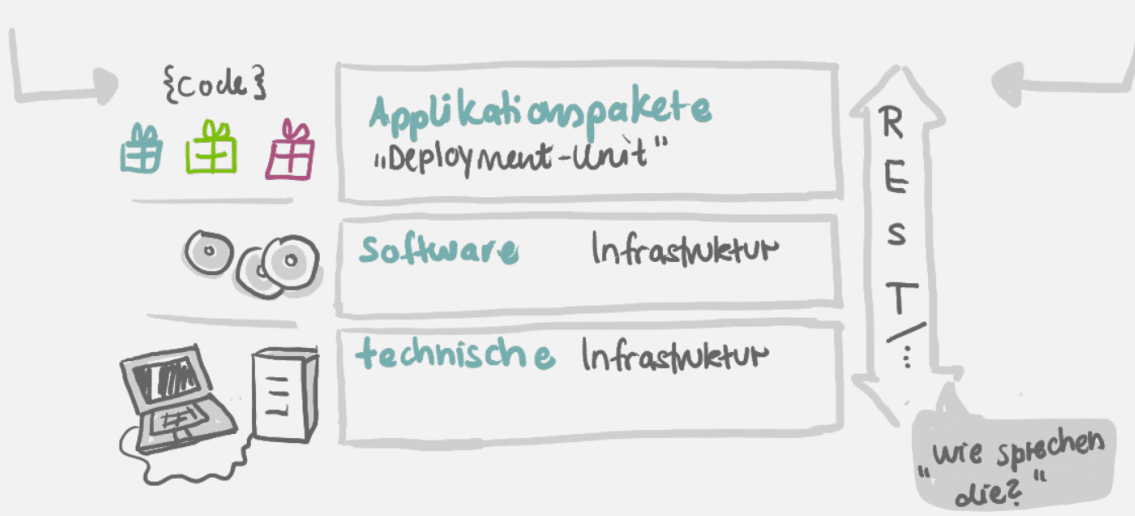


QA|WARE

TI-ARCHITEKTUR



??



Schritte

Schichten?



Daten?
↳ NoSQL
↳ SQL



Rechner



Sicherheit



Verteilung Software-Infrastruktur



Wo läuft was?



Backendtechnologie?



Verbindungen



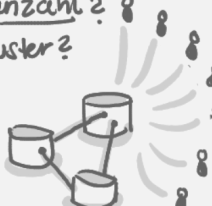
Verfügbarkeit



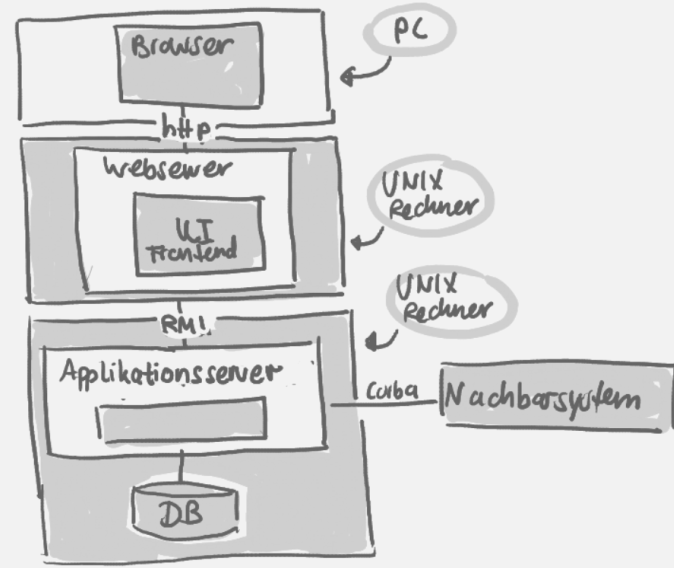
Balancing



Nutzeranzahl? ?
↳ cluster?



Beispiel



T-ARCHITEKTUR

was um A drumrum?

wie A auf TI ausgeführt?

Schichten

welche?

Technologien?
(OpenSource Bausteine!)

JavaFX

EJB

JPA

Strikt



nonstrikt



nie nach
OBEN



!!!

Software BLUTGRUPPEN



O

"Bestimmt von
Gonichts"

} String
Date

A

fachlich "eigentliches Projektziel"

I want...



T

"nicht fachlich, nicht weltwissen"

MyClass
MyInterface

AT

"booo!" Mischung aus A & T



R

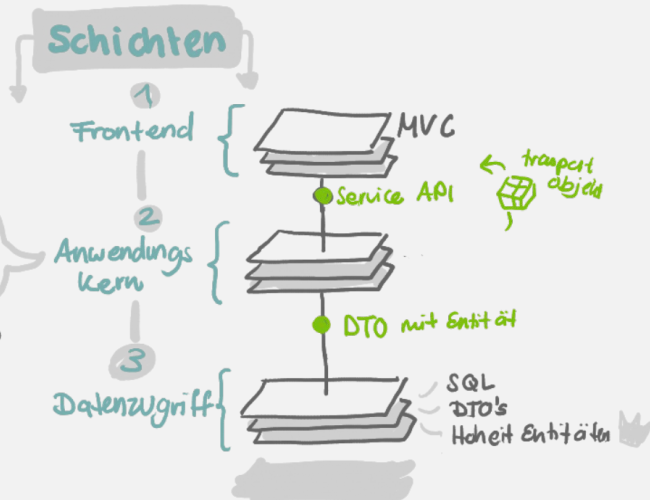
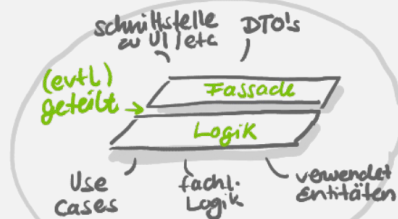
"Weichmacher" Kategorie $\approx$ AT $\rightarrow$ mehr Vermittler

DAMIT
SPIELEN

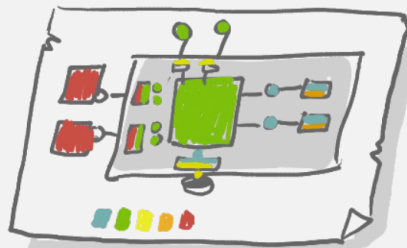


QA|WARE

Detail



HowTo?



- Kern : möglichst A 
- Fremdsysteme? → Adapter bauen
- Komponenten: viel !
- DB Anbindung: $\emptyset$ übersetzt zu T 



QAWARE

I-ARCHITEKTUR

Patterns


Regeln { }


Paket-Struktur


Module


Micro Design


Abhängigkeiten

„wer spricht mit wem?“ ...

KOMPONENTEN

MODULE

Konvention:
1-zu-1



common

config

web

...

brauchte mehr Dynamik???

Schnittstellen können auch in
separate Module gepackt
werden!



QAWARE



QAWARE

Module



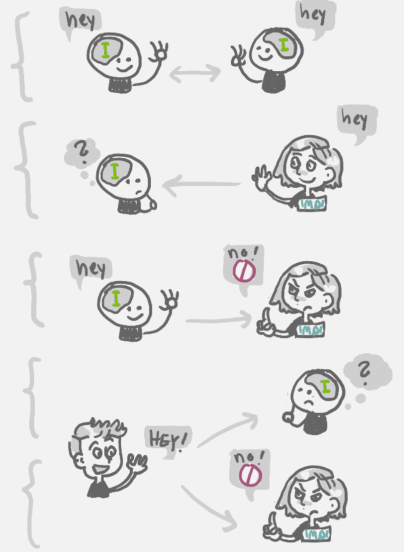
pro Komponente 1 Modul
↳ Schnittstelle an oberster Stelle
↳ Implementierung getrennt



...von innen...



Abhängigkeitsregeln



keine cyclen!



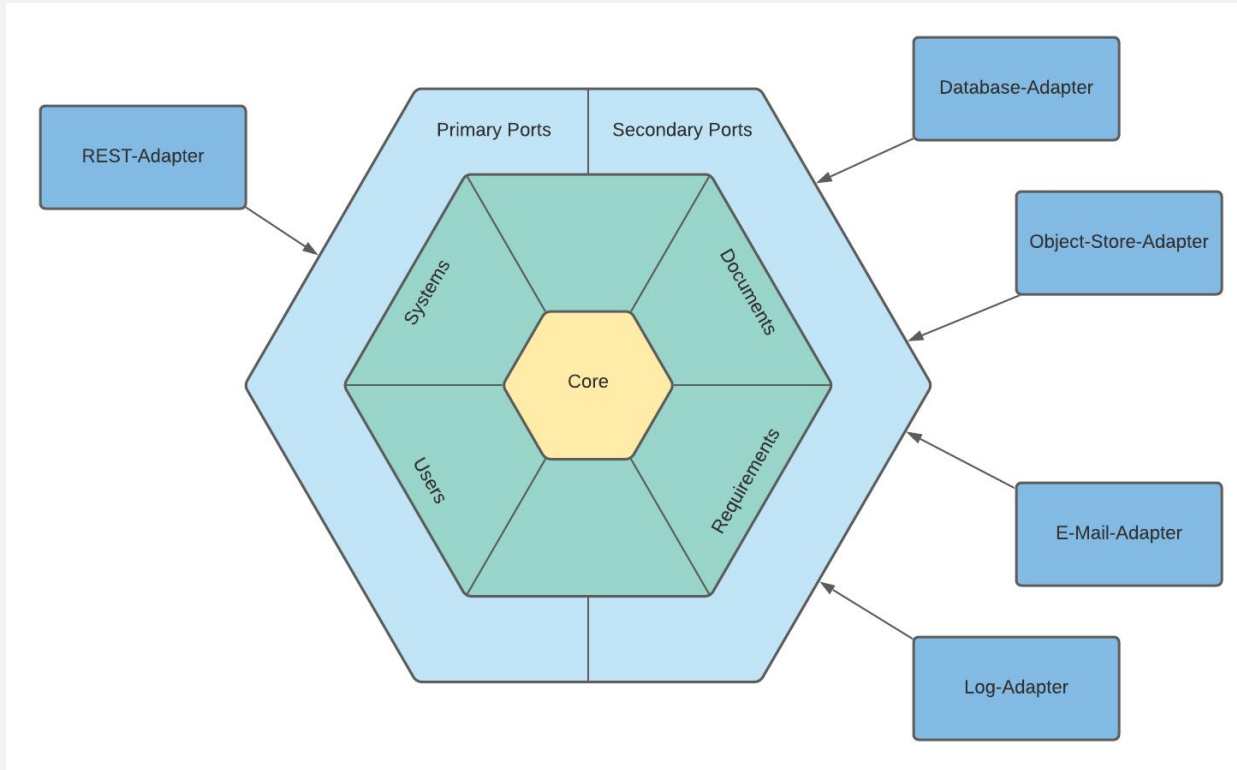
QA|WARE

Hexagonale Softwarearchitektur

Hexagonale Architektur löst viele der beschriebenen Probleme.



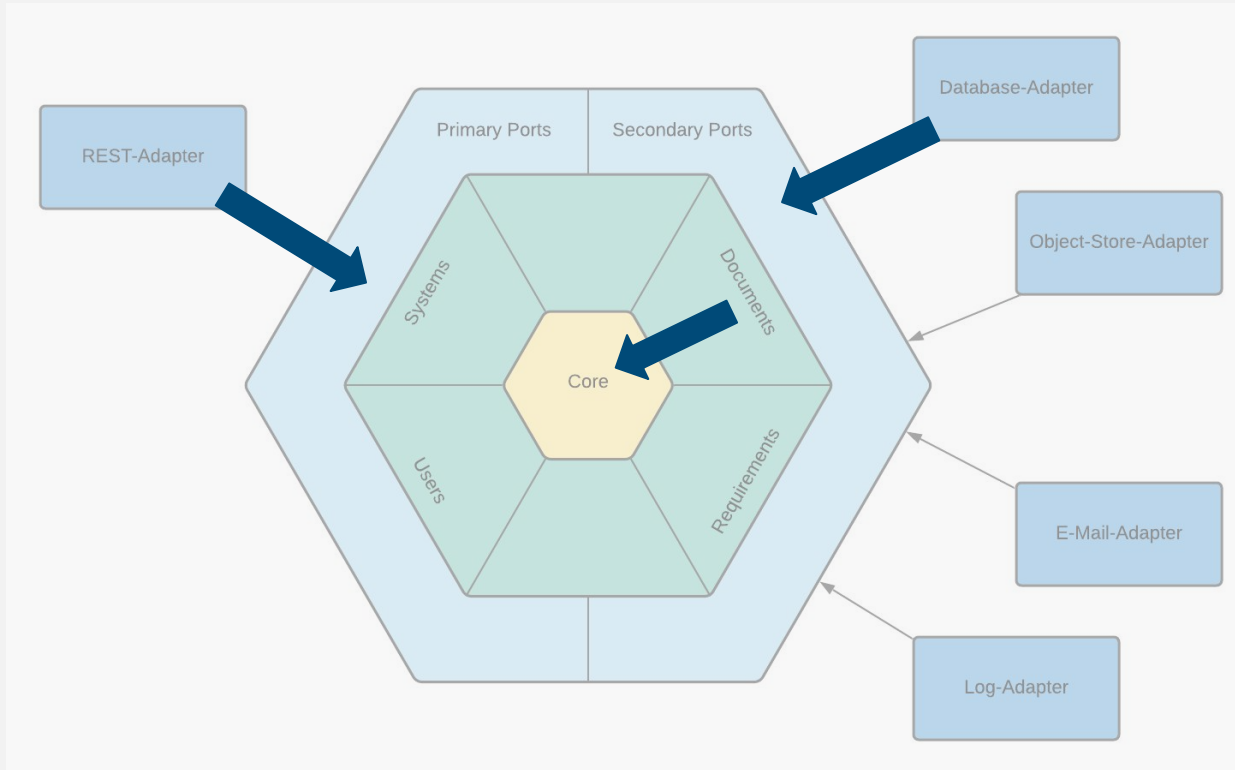
QA|WARE



Hexagonale Architektur löst viele der beschriebenen Probleme.



QA|WARE





QA|WARE

**Welche Architektur verwendet ihr
in eurer Projektarbeit?**



QA|WARE

Aber welche Architektur ist die richtige für mich?

What the **** habe ich mir dabei gedacht?!



QA|WARE



Oft geht Entscheidungskontext im Projektalltag verloren.



QA|WARE

- Beim Programmieren treffen wir durchgehend Entscheidungen:
 - welche Programmiersprache?
 - wie benenne ich meine Variablen
 - if oder switch-case
 - welchen Datentyp verwende ich
- Wenn wir später wieder an derselben Stelle vorbeikommen, fragen wir uns oft, wieso wir genau diese Entscheidung getroffen haben

Oft geht Entscheidungskontext im Projektalltag verloren.



QA|WARE

- Codestellen geprägt von mehreren Entwicklern:innen
- Entwickler:innen verlassen das Projekt
- Entscheidungen werden zu schnell getroffen, ohne an die Konsequenzen zu denken → big picture fehlt!
- Kolleg:innen haben nicht genügend Erfahrung, um richtige Entscheidungen zu treffen

Mit Architecture Decision Records (ADR) lassen sich Architekturentscheidungen systematisch dokumentieren.



QA|WARE

- Ein ADR ist ein point-in-time Dokument, das eine Architekturentscheidung und die Begründung dahinter festhält
- Dabei gibt ein ADR Aufschluss zu folgenden Fragen:
 - Worüber wurde entschieden?
 - Was ist der Status der Entscheidung?
 - In welchem Kontext wurde entschieden?
 - Was war die Entscheidung?
 - Welche Konsequenzen ergeben sich aus der Entscheidung?
 - optional: Wer war involviert?

ADRs haben viele Vorteile für das Projektteam.



QA|WARE

- Zeit- und Kostenersparnis
- Verteilte Entscheidung
- Transparenz von Entscheidungen
- Wissenstransfer
- Klare Verantwortlichkeiten

Braucht es für meine Entscheidung wirklich ein ADR?



QA|WARE

- Wie immer solltet ihr euch die Frage Stellen: Ist die Architekturentscheidung relevant genug, um dokumentiert zu werden? Eine Entscheidung ob if oder switch-case macht eher weniger Sinn.
- ADRs sollten v.a. für Entscheidung verwendet werden, die eine langfristige Auswirkung auf euer Projekt haben

→ Wenn ihr die Notwendigkeit eines ADRs identifiziert habt, erstellt ein **ADR im Entwurfsstatus**

Zu welchem Problem möchte ich eine Entscheidung treffen?



QA|WARE

- Definiert genau das Problem, dass ihr mit eurem ADR adressieren wollt
- Hilft anderen zu verstehen, warum für diese Entscheidung ein ADR notwendig war

→ Definiert den **Kontext**, in dem der ADR erstellt wurde

Welche Möglichkeiten gibt es für das beschriebene Problem?



QA|WARE

- Definiert genau welche Alternativen bei der Entscheidungsfindung in Betracht gezogen wurden
- Notiert für alle Alternativen die Vor- und Nachteile
- Dies hilft Kolleg:innen dabei den Entscheidungsprozess auch nachträglich noch nachzuvollziehen

→ Gibt Aufschluss über den **Entscheidungsprozess**

Diskutiert im Team für welche Lösung ihr euch entscheidet.



- Arbeitet hier mit allen Stakeholdern und Entscheidungsträgern zusammen
- Diskutiert über Alternativen sowie Vor- und Nachteile der Optionen
- Getroffene Entscheidung sollte mit den Stakeholdern (bspw. Kunden) abgesprochen werden

→ Jetzt habt ihr eine **Entscheidung** für euer anfängliches Problem

Falsche Dokumentation ist schlimmer als keine Dokumentation.



QA|WARE

- ABER: ADRs können ihre Gültigkeit verlieren
- Beispiel: Wir haben uns für eine MySQL Datenbank entschieden, aber Stellen fest, dass wir plötzlich Binärdokumente speichern müssen → Wir brauche einen S3-Store
- Wenn ein ADR **A** seine Gültigkeit verliert, ändert den Status in “Veraltet durch **B**”
- Erstellt einen neuen ADR mit dem Status “Löst **A** ab”

Beispiel eines ADR.



QA|WARE

Title	001: Decision to Choose a Cloud Provider for Data Project
Status	Accepted
Context	Our project involves developing a data-intensive application requiring a reliable and scalable cloud infrastructure. We must decide among Azure, AWS, and GCP as potential cloud providers. Options Considered: • Azure (Microsoft Azure): Offers a wide range of services, strong integration with Microsoft products, and a growing ecosystem. • AWS (Amazon Web Services): Known for its service offerings, mature infrastructure, and a large customer base. • GCP (Google Cloud Platform): Provides advanced data analytics capabilities, machine learning services, and Google's global network infrastructure. Quality requirements or non functional we had to consider includes performance, scalability, ease of use, pricing, integration, support, and documentationl

Beispiel eines ADR.



QA|WARE

Decision	After thorough evaluation, we choose Google Cloud Platform (GCP) due to its alignment with our quality attributes and long-term viability. Rationale: <u>Quality Attributes Analysis</u> • Performance and Scalability: GCP's global network infrastructure and focus on data processing align well with our application's needs for scalability and responsiveness. • Ease of Use: GCP's user-friendly interface and intuitive navigation are likely to reduce the learning curve for our team, resulting in quicker adoption. • Pricing: A thorough cost analysis indicates that GCP's pricing structure suits our use case and offers potential cost savings through sustained usage discounts. • Integration: GCP's open API approach provide flexibility and minimize vendor lock-in, aligning with our desire for a more adaptable infrastructure. <u>Long-Term Viability</u> • GCP's continued growth, established technology stack, and reputation in the industry provide confidence in its ability to support our project's needs over the long term. • Our Innovation team built a scalable and low cost application on GCP
Consequences	• Positive Impact: GCP's robust data analytics, machine learning services, competitive pricing, and user-friendly interface are expected to positively influence our application's development, performance, and overall success. • Potential Drawbacks: Migrating to GCP will require dedicated effort from our technical team to ensure seamless integration, performance and optimisation with GCP's services. • Monitoring and Adjustment: We will monitor our application's performance and associated costs on GCP closely. If changes in requirements or unforeseen challenges arise, we will be prepared to revisit our decision.

Es gibt mehrere verschiedene ADR Formate.



QAWARE

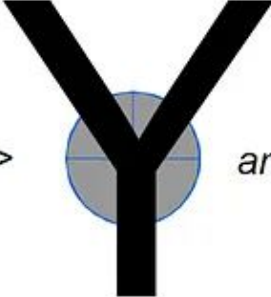
- Von oben: NygardADR
 - MADR: Markdown ADR
(<https://github.com/adr/madr/blob/4.0.0/template/adr-template.md?plain=1>)
 - Y-Statement
 - Beliebige andere ADR Templates
 - weitere Infos und Tools gibt's unter: <https://adr.github.io/>
 - Für eure Projektarbeit könnt ihr euch hier nochmal alle Infos holen:
<https://github.com/joelparkerhenderson/architecture-decision-record?tab=readme-ov-file#how-to-start-using-adrs-with-git>
- nutzt das Format, das euch am meisten zusagt und zu eurem Projekt passt

Beispiel eines (WH)Y-Statements.



QA|WARE

*In the context of <use case uc
and/or component co>, ... facing <non-functional concern c>,
.. we decided for <option o1> and neglected <options o2 to on>,
... to achieve <quality q>,
... accepting downside <consequence c>.*





QA|WARE

Architektur dokumentieren

Problem: StackOverflow driven development



QA|WARE



*“[...] developers use **Stack Overflow** as a **major resource** for **solving programming problems** [...]*

*Other resources, such as **official Android API documentation**, **do not provide** the same degree of **quickly understandable**, directly applicable **assistance**. Our results suggest that using Stack Overflow helps Android developers to arrive at functional solutions more quickly than with other resources. [...]*

*Because **Stack Overflow** contains **many insecure answers**, Android developers who rely on this resource are **likely to create less secure code**.”*

- You Get Where You're Looking for: The Impact of Information Sources on Code Security 2016 (10.1109/SP.2016.25)

Security and Compliance



QA|WARE

*“Documentation is also **critical**.*

*[...] this suggests the **importance** of **creating official documentation** that is **useful enough** to **keep developers from searching out unvetted**, potentially insecure alternatives.*

*[...] the **excellent code examples** for PyNaCl and in the cryptography.io “recipes” layer appear to have **contributed to high rates of security success**.”*

- Comparing the Usability of Cryptographic APIs 2017 (10.1109/SP.2017.52)



reddit.com

reddit

Suche in r/golang

Anmelden

Einstellungsmenü öffnen

← r/golang • vor 2 Jahren
haaslelin

What do you like about Go?

discussion

I am a C#/Angular developer that loves the Microsoft stack. I believe they go a great direction nowadays and I enjoy C#, Typescript as well as the tooling around it.

But I could not stop to notice all the gopher stickers on the laptops of presenters on conferences. After looking up Go I have the impression that it is liked quite a lot. So I am asking you, the Go community, what things do you like most about Go? And maybe also what you don't like about it if there is anything.

65 141 Teilen Melden

+ Kommentar hinzufügen

Sortieren nach: Beste

Suche Kommentare durchsuchen

DARKHUMOR-D • vor 2 Jahren

- Documentation is awesome
- Runs fast
- Super easy to learn
- Tooling is great
- My favourite: Concurrency is actually a joy to work with, no funny business. Goroutines are so cool. I built a bittorrent client in go, with concurrent networking, and it was so easy to write and debug. I recommend [Jake Wright's video on concurrency](#) in Go if you want a quick peek.

Go feels fun and simple to write, even when you're building really complex stuff, once you get going it really doesn't get in your way.

45 Antworten Auszeichnen Teilen

Why Golang gives me a prefix certain comment?
3 Upvotes · 6 Kommentare

r/golang

What is it about the Go language that makes it attractive for distributed...
91 Upvotes · 35 Kommentare

r/golang

How do you handle Sets?
17 Upvotes · 70 Kommentare

r/golang

Go 1.14 Release Notes
257 Upvotes · 29 Kommentare

r/golang

What "sucks" about Golang?
r/baduk

What's the best way to learn Go online for free, starting from zero?
26 Upvotes · 43 Kommentare

r/golang

When do you say "Go"? "golang"?
4 Upvotes · 15 Kommentare



QA|WARE

Good documentation is an enabler for successful products



QA|WARE

Every Product needs Documentation

Imagine
assembling
furniture without a
manual.



42

docs.spring.io

Why Spring ▾ Learn ▾ Projects ▾ Academy ▾ Solutions ▾ Community ▾ ⚙️

Spring Boot 3.3.4

Q Search ⌘ + k

- Overview
- Documentation
- Community
- System Requirements
- Installing Spring Boot
- Upgrading Spring Boot
- > Tutorials
- > Reference
- > How-to Guides
- > Build Tool Plugins
- > Spring Boot CLI
- > Rest APIs
- > Java APIs
- > Kotlin APIs
- > Specifications
- > Appendix

Spring Boot / How-to Guides / Spring MVC

Spring MVC

Spring Boot has a number of starters that include Spring MVC. Note that some starters include a dependency on Spring MVC rather than include it directly. This section answers common questions about Spring MVC and Spring Boot.

Write a JSON REST Service

Any Spring `@RestController` in a Spring Boot application should render JSON response by default as long as Jackson2 is on the classpath, as shown in the following example:

```
Java Kotlin
```

```
@RestController
public class MyController {

    @RequestMapping("/thing")
    public MyThing thing() {
        return new MyThing();
    }

}
```

JAVA

Spring MVC

- Write a JSON REST Service
- Write an XML REST Service
- Customize the Jackson ObjectMapper
- Customize the `@ResponseBody` Rendering
- Handling Multipart File Uploads
- Switch Off the Spring MVC `DispatcherServlet`
- Switch Off the Default MVC Configuration
- Customize `ViewResolvers`

- Edit this Page
- GitHub Project
- Stack Overflow

Example



QAWARE

K8s Platform Guide

1. Getting Started
2. Security
3. Continuous Delivery
4. CI/CD Includes
5. CLI
6. ArgoCD
7. Prometheus
8. Traefik
9. OAuth2 Proxy
10. Velero
11. Terraform
12. User Modules



Container Platform User Guide

1. Deploy your first App
2. Concepts
 - a. What is a <Product Name>?
 - b. How is it Secured?
 - c. How is it developed? (Continuous Delivery)
 - d. Everything as Code (ArgoCD, Terraform)
3. How To
 - a. Deploy Apps (GitLab CI, ArgoCD)
 - b. Connect to the Platform (CLI)
 - c. Monitor Applications (Prometheus)
 - d. Expose Applications (Traefik, OAuth2)
 - e. Setup Automatic Backups (Velero)
 - f. Add a Database (Terraform, User Modules)



Use fitting Language

Use appropriate Structure

Know your Audience

Typical Audiences in Software Projects

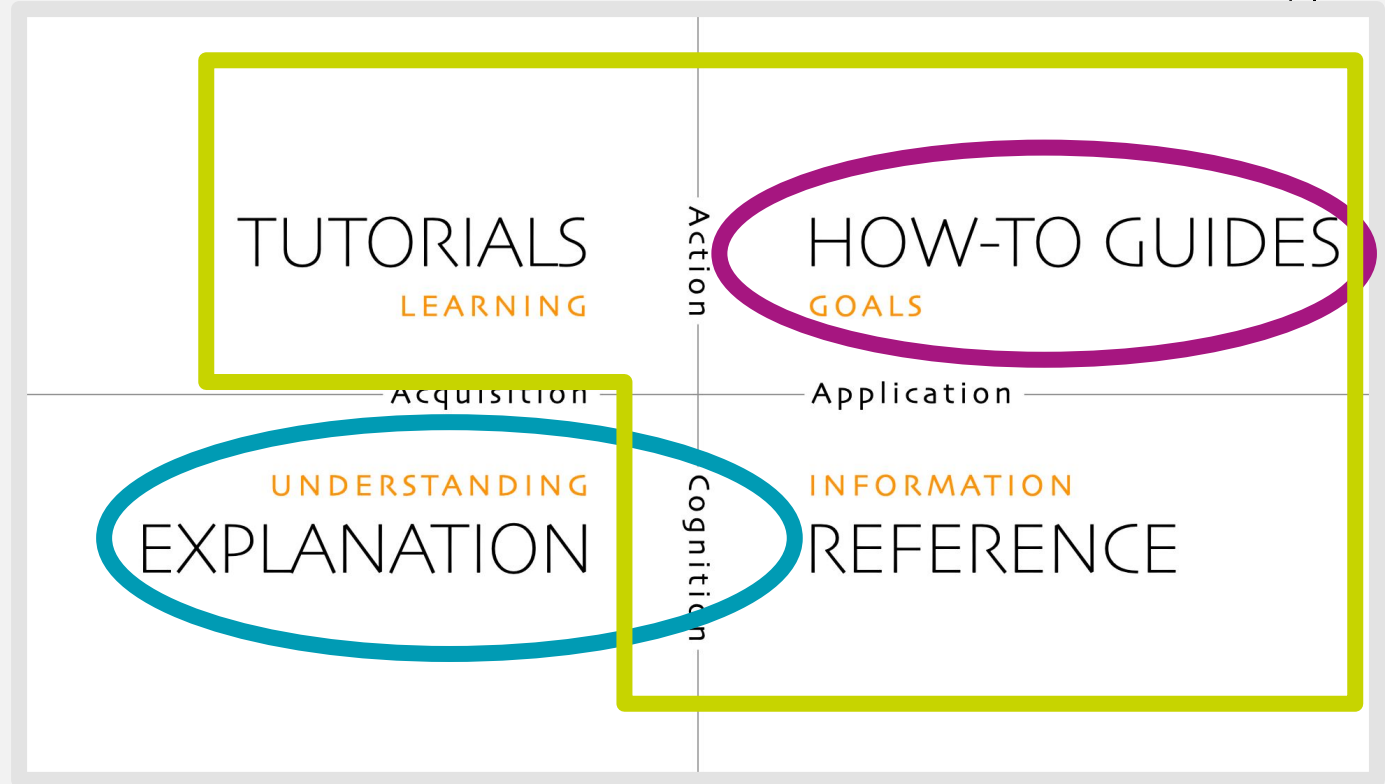


QA|WARE

Developers

Architects

Users



Mit arc42 lassen sich Softwarearchitekturen strukturiert dokumentieren.



QA|WARE

arc42 gibt praxisorientierte Antworten auf die folgenden Fragen:

1. **Was** sollen wir über unsere Architektur kommunizieren/dokumentieren?
2. **Wie** sollen wir kommunizieren/dokumentieren?

arc42 hat genau dazu ein Template mit 12 Kapiteln entwickelt, in denen sich alles Wissenswertes zur Architektur unterbringen lässt.

arc42 setzt sich aus 12 Abschnitten zusammen.



QA|WARE

1. Einführung & Ziele

Grundlegende Anforderungen, insbesondere Qualitätsziele

2. Randbedingungen

Regelungen und externe Randbedingungen

3. Kontext & Abgrenzung

Externe Systeme und Schnittstellen

4. Lösungsstrategie

Kernideen und Lösungsansätze

5. Bausteinsicht

Aufbau des Quellcodes, Modularisierung (hierarchisch)

6. Laufzeitsicht

Wichtige Laufzeitszenarien

7. Verteilungssicht

Hardware, Infrastruktur & Deployment

8. Querschnittliche Konzepte

Querschnittsthemen, oft sehr technisch und detailliert

9. Architekturentscheidungen

Wichtige Entscheidungen (nicht anderweitig beschrieben)

10. Qualitätsanforderungen

Qualitätsbaum, Qualitätsszenarien

11. Risiken & Technische Schulden

Bekannte Probleme und Risiken

12. Glossar

Wichtige und spezifische Begriffe ("gemeinsame Sprache")

1. Einführung und Ziele



QA|WARE

- Beschreibt alle relevanten Anforderungen und antreibende Faktoren, die das Entwicklerteam beachten muss
- Dabei handelt es sich v.a. um folgende Aspekte:
 - Essenzielle Funktionen der Anwendung
 - Qualitätsziele der Software
 - Relevante Stakeholder und deren Erwartungen
 - Wirtschaftliche Ziele

2. Randbedingungen



QA|WARE

- Beschreibt alle relevanten Einschränkungen, die das Team bei der Implementierung der Architektur beachten muss
- Einschränkungen sind manchmal auch außerhalb des Projektes für die gesamte Organisation gültig
- Beantwortet die Frage, wo genau der Architekt selbst entscheiden kann und wo Entscheidungshoheit bei jemand anderem liegt
- Beispiel: Konzern ist AWS Partner und will daher Microservices nur in der AWS Cloud deployen

3. Kontextabgrenzung



QA|WARE

- Grenzt das eigene System von externen Kommunikationspartnern ab:
 - Nutzer:innen
 - Nachbarsysteme
- Spezifiziert externe Schnittstellen aus fachlicher Sicht (immer!)
- Im Zuge der Schnittstellendefinition können auch bereits domänenspezifische Formate und Kommunikationsprotokolle definiert werden

4. Lösungsstrategie



QA|WARE

- Kurze Zusammenfassung und Erläuterung der grundlegenden Entscheidungen und Lösungsstrategien, die die Architektur beeinflussen
- Hierfür könnt ihr euch in Teilen an euren ADRs bedienen
- Beinhaltet v.a.:
 - Technologieentscheidungen
 - Entscheidungen zur Architektur im Großen
 - Entscheidungen zur Einhaltung von key-Qualitätsanforderungen
 - Relevante organisatorische Entscheidungen

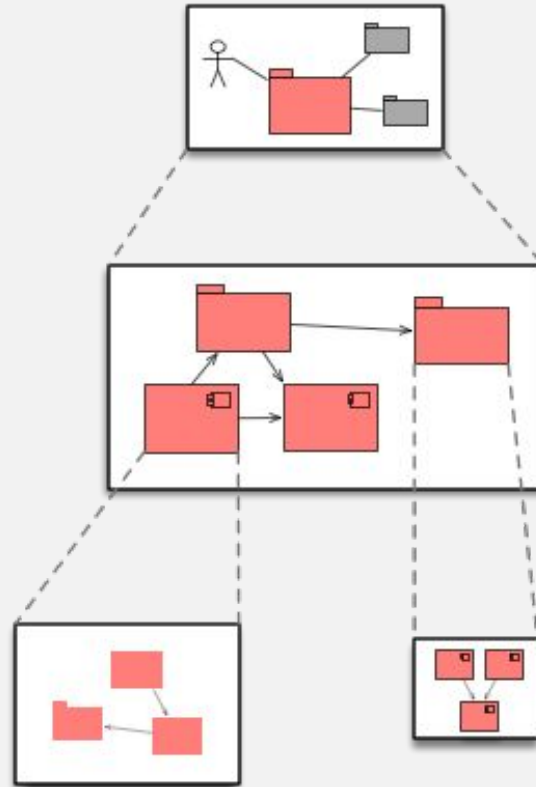
5. Bausteinsicht



QA|WARE

- Statische Zerlegung des Systems
- Die Abstraktion des Source-Codes dargestellt als Hierarchie von “White-Boxes”, die wiederum kleine Black-Boxes enthalten
- Dabei können folgende Elemente entstehen
 - Module und Komponenten
 - Klassen und Interfaces
 - Frameworks
 - Schichten
 - usw.
- Für diesen Schritt eignen sich insbesondere Diagramme (bspw. UML)
- Zeigt eure Architektur aus der T-Sicht

5. Bausteinsicht



* Hierzu werden wir uns in einer der folgenden Veranstaltungen noch einmal ein weiteres Modell ansehen

6. Laufzeitsicht



QA|WARE

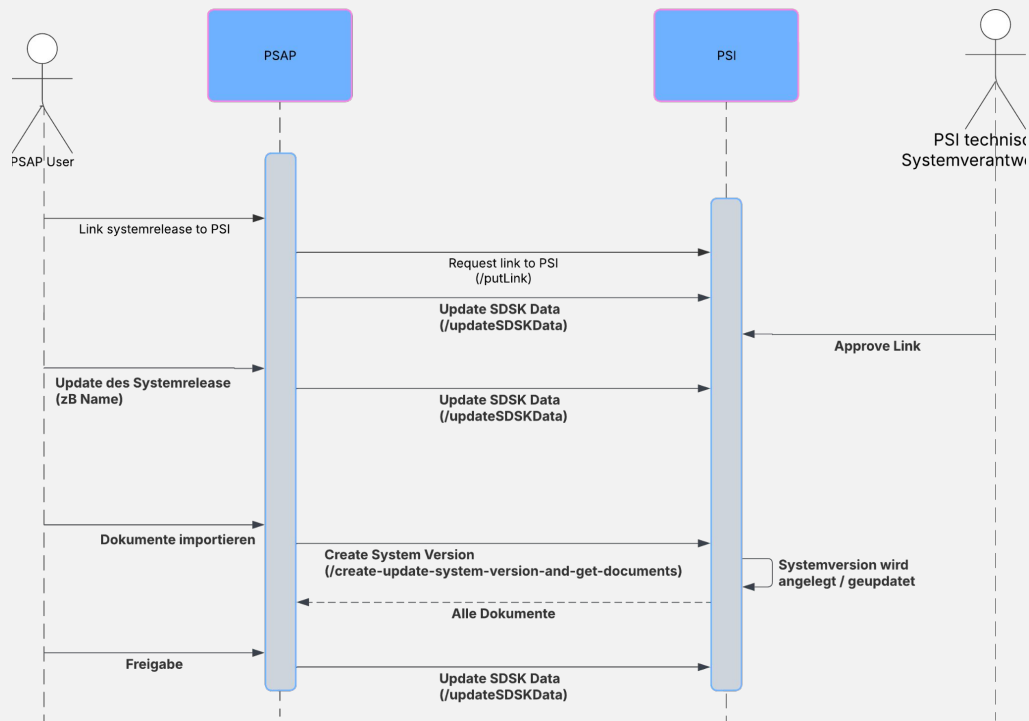
- Beschreibt das Verhalten von Bausteinen in Form von dynamischen Szenarien
- Kann z.B. der Flowchart eines Zahlungsprozesses sein (Wann wird die Bank angefragt, wann bekommt der Kunde Rückmeldung etc.)
- Darstellung häufig in Form von Flow-Charts mit Zeitachse
- Hier werden besonders wichtige Features und Prozesse abgedeckt
- Auch kritische Ausnahmen und Fehlerfälle können modelliert werden
- Wichtige Projektprozesse (Deployment etc.)
- Interaktion mit kritischen Externen Interfaces (* Auch dazu später mehr)

6. Laufzeitsicht



QA|WARE

Standardfall



7. Verteilungssicht



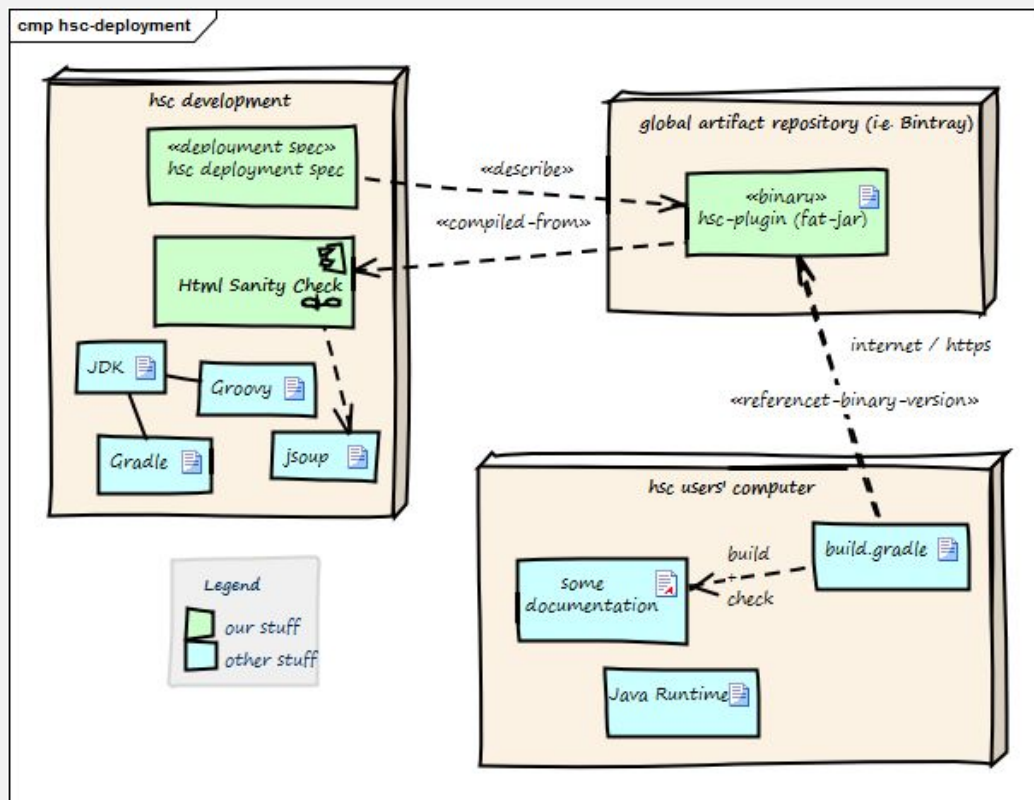
QA|WARE

- zeigt die technische Infrastruktur mit echten oder virtuellen Prozessoren
- zeigt die Systemtopologie und die Abbildung der Software-Bausteine auf dieser Infrastruktur
- die einzelnen Prozessoren werden genau beschrieben mit:
 - Geographische Position
 - Umgebung (Linux etc.)
 - Maschinen-Specs
- alle relevanten Umgebungen dokumentieren! (Testing, Production etc.)
- zeigt eure “Architektur im Großen” aus der TI-Sicht

7. Verteilungssicht



QAWARE



8. Querschnittliche Konzepte



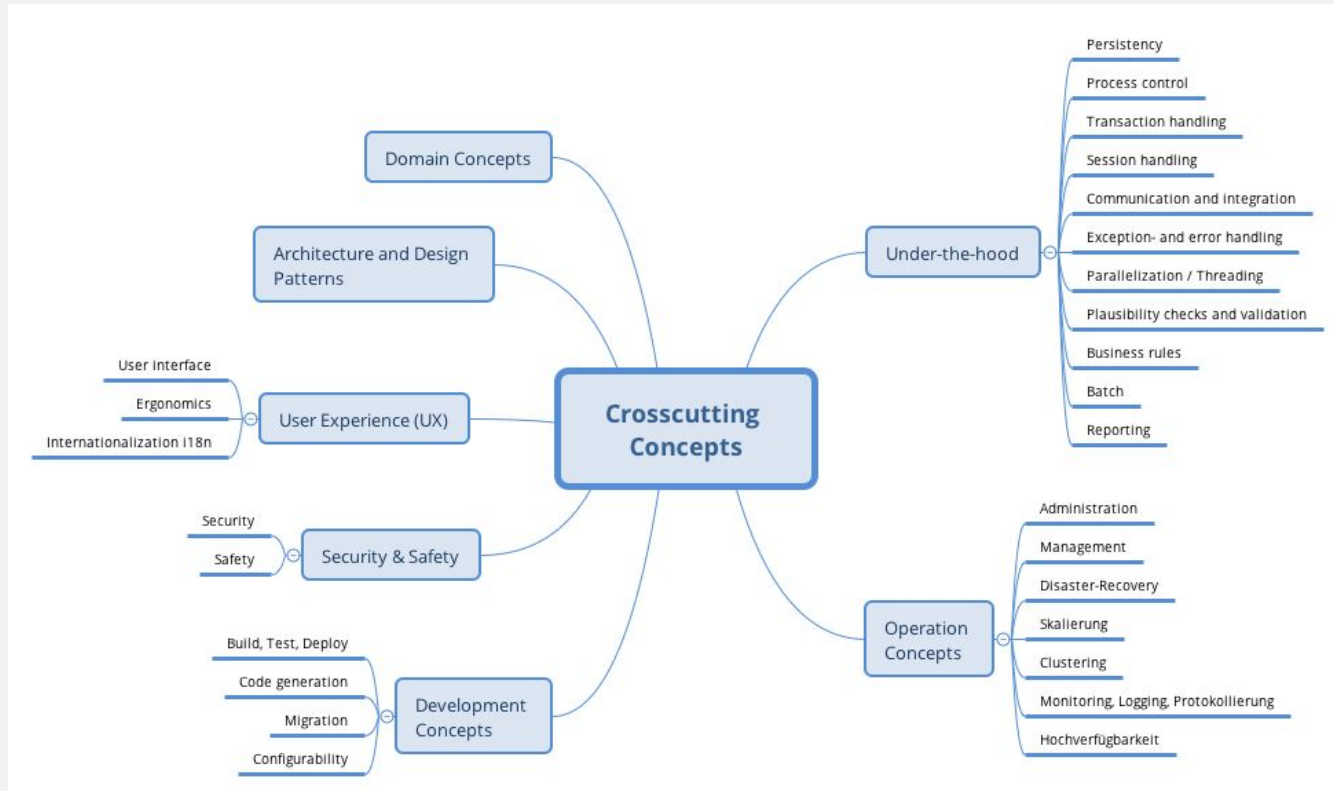
QA|WARE

- beschreibt allgemein gültige Prinzipien, Lösungsansätze und Ideen
- diese sind für mehrere Stellen in der Anwendung gültig
- Beispiele hierfür können sein:
 - Codestil
 - Kommentarstil
 - Architektur- und Designpatterns
 - Regeln zur Nutzung bestimmter Technologien
 - grundsätzliche technische Entscheidungen

8. Querschnittliche Konzepte



QA|WARE



9. Architekturentscheidungen



QA|WARE

- wichtige, teure, kritische oder riskante Architekturentscheidungen, die zentrale Bedeutung für das Gesamtsystem haben
- Begründung der Entscheidungen ist hier aufgeführt
- Hier sollten nur “wichtige” Entscheidungen als ADRs aufgeführt sein
- Entscheidungen im Kleinen sollten direkt in Kapitel 5 an den einzelnen Bausteinen dokumentiert sein
- Hier kann auch gerne noch einmal auf Kapitel 4 referenziert werden, wo Architekturentscheidungen gemacht wurden (aber noch ohne Begründung)

→ Es gilt nach wie vor: Nur ADR, wenn wirklich notwendig!

10. Qualitätsanforderungen



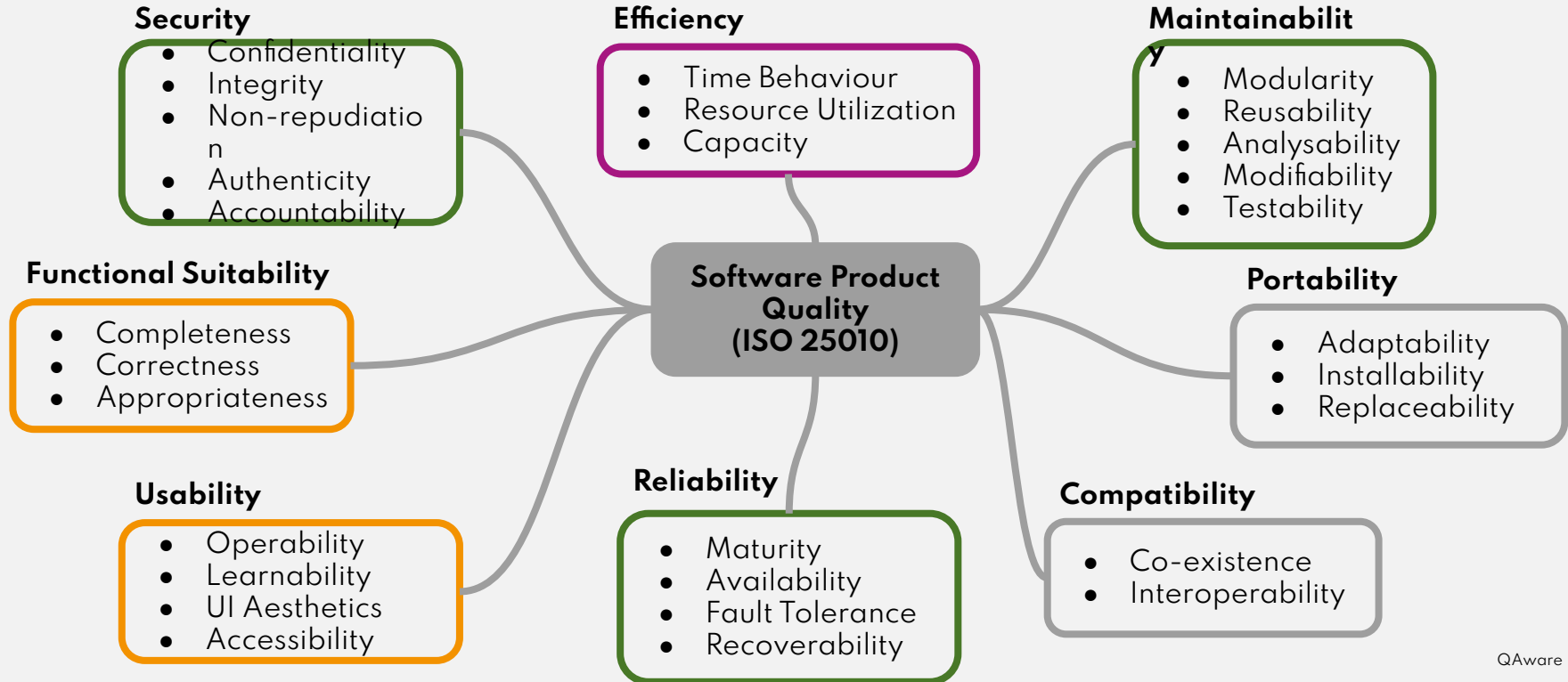
QA|WARE

- noch einmal ein Deep-Dive in Anforderungen an die Qualität des Projektes
- die wichtigsten Anforderungen sollten schon in Kapitel 1 aufgeführt worden sein
- dieses Kapitel beinhaltet auch weniger wichtige Qualitätsziele der Anwendung
- Beispiele können sein:
 - Pageload und First Contentful Paint
 - SEO
 - Multilanguage

Die ISO 25010 bietet einen guten Überblick über potentielle Qualitätsziele.



QA|WARE



11. Risiken und technische Schulden



QA|WARE

- listet alle technischen Risiken und Schulden auf
- diese Liste ist nach Priorität geordnet
- technische Risiken können sein:
 - Datenbankbottleneck
 - System basiert auf Technologie, die eventuell in Zukunft nicht mehr unterstützt wird
- fachliche Risiken können sein:
 - Konkurrenz baut ähnliche Features und löst unser Produkt ab
 - Software funktioniert nur mit den aktuellen Anforderungen und skaliert nicht

12. Glossar



QA|WARE

- das klassische Glossar bpsw. in Tabellenform
- beinhaltet wichtige Domänenbegriffe, die alle Stakeholder kennen sollten
- schafft eine gemeinsame Gesprächs- und Diskussionsgrundlage, wenn man über das Projekt spricht
- enthält auch manchmal Übersetzungstabellen, wenn in mehrsprachigen Teams gearbeitet wird

Noch einmal ein Überblick über arc42.



QA|WARE

1. Einführung & Ziele

Grundlegende Anforderungen, insbesondere Qualitätsziele

2. Randbedingungen

Regelungen und externe Randbedingungen

3. Kontext & Abgrenzung

Externe Systeme und Schnittstellen

4. Lösungsstrategie

Kernideen und Lösungsansätze

5. Bausteinsicht

Aufbau des Quellcodes, Modularisierung (hierarchisch)

6. Laufzeitsicht

Wichtige Laufzeitszenarien

7. Verteilungssicht

Hardware, Infrastruktur & Deployment

8. Querschnittliche Konzepte

Querschnittsthemen, oft sehr technisch und detailliert

9. Architekturentscheidungen

Wichtige Entscheidungen (nicht anderweitig beschrieben)

10. Qualitätsanforderungen

Qualitätsbaum, Qualitätsszenarien

11. Risiken & Technische Schulden

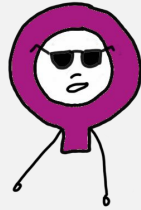
Bekannte Probleme und Risiken

12. Glossar

Wichtige und spezifische Begriffe ("gemeinsame Sprache")



QA|WARE



arc42 is cool

- always know what to write
- never forget something
- structure is familiar over time



QA|WARE

Wie genau schreibe ich meine Dokumentation?

5C's for Technical Writers



QA|WARE

Clarity

Clear text and information.

Concise

Keep it short. Only provide required information.

Consistent

Use the same word throughout the whole document.

Complete

Give all required information but keep **Concise** in mind.

Correct

Mistakes and typos are bad for the users and your trust.

Start small



QA|WARE

A **README.adoc** is **all you need** to start your journey to a great documentation!

```
= My project  
// Tell me what is the project about
```

```
== Installation  
// How can I install/use it
```

```
== Configuration  
// What do I need to configure to use  
it
```

```
== Contribute  
// How can I contribute?
```

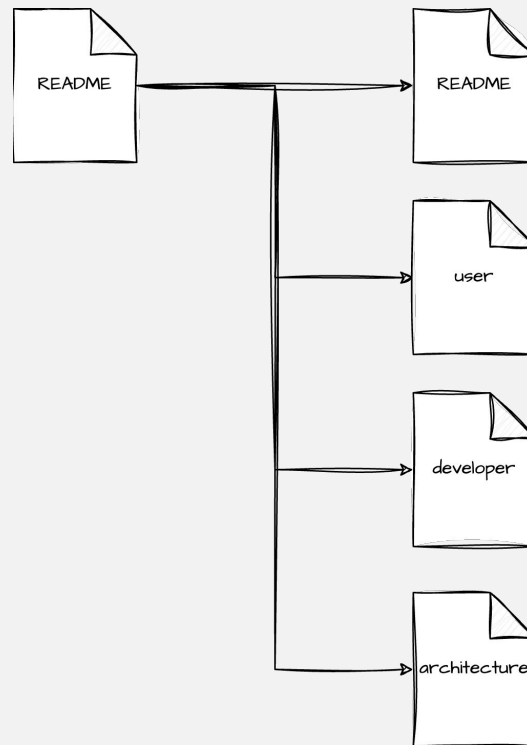
Grow as you go



QA|WARE

As your README.adoc **grows with content** you want to **split it up**:

- **README.adoc**
⇒ Short (marketing) intro and links to the real docs
- **docs/user.adoc**
⇒ Docs that will be read most often. Written for the actual users.
- **docs/developer.adoc**
⇒ Docs written for developers of the project.
- **docs/architecture.adoc**
⇒ A reference to understand the architecture and decisions made.



Final stage: Modules

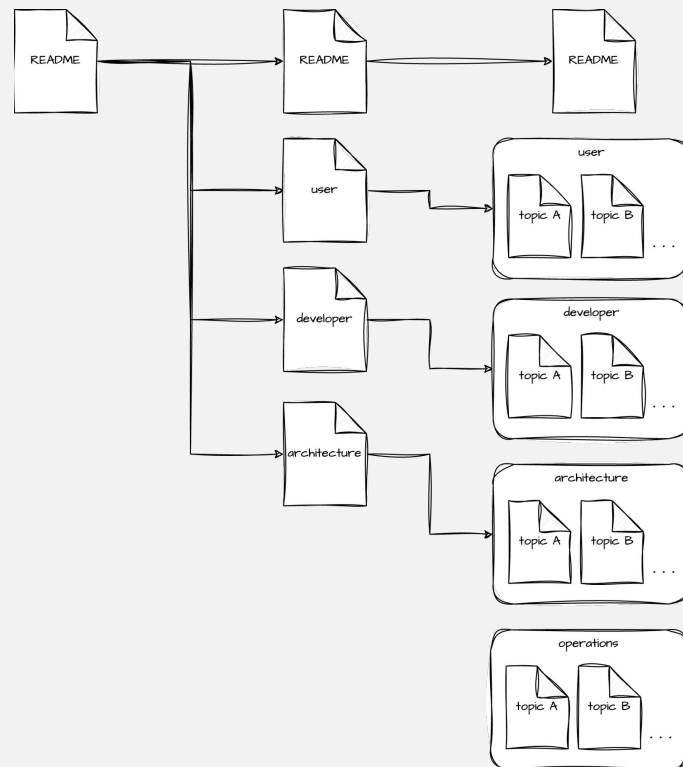


QA|WARE

As your **project scales** the **documentation grows**.

To maintain a **very good UX** when accessing the docs
Modules helps to **structure and display** your **content**.

Caveat: Things get **harder to find** as the amount of documentation grows.



Grow organically



QA|WARE

Write down what you are **doing** right **now**. See where it fits and adapt the docs and technology appropriately.

⚠ What you *don't* want to do, is to just fill some schema with content that has no purpose.



QA|WARE

Key Learning: Grow step by step

Theorie schön und gut, aber in welchem Format machen wir das?



QA|WARE

- Auch hier gilt: it depends! Üblicherweise bestehen aus den folgenden Bausteinen:
 - Verschiedene Textbausteine
 - Tabellen
 - Grafiken (häufig UML-Diagramme und Flowcharts)
- Nutzt dafür das Tool, mit dem ihr am besten arbeiten könnt
- Templates in verschiedenen Formaten findet ihr hier:
<https://arc42.org/download#file-based-formats>



QA|WARE

Which markup (language) should I use?



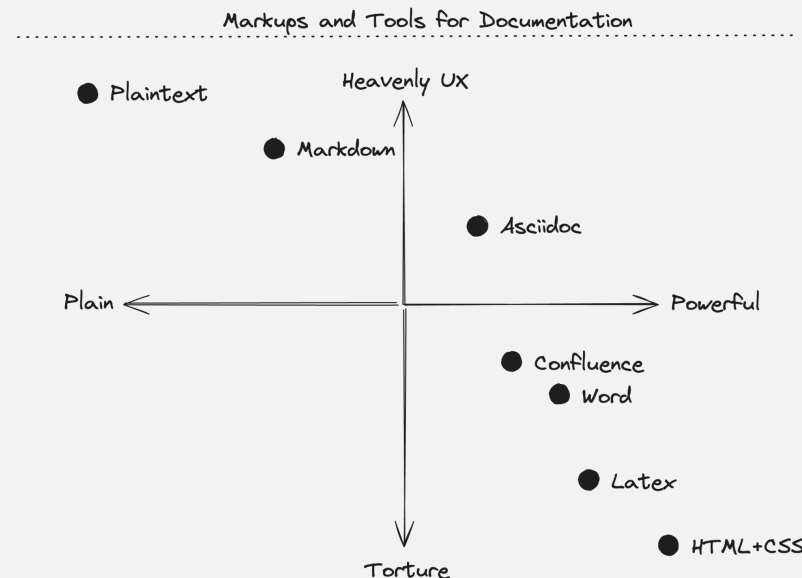
AsciiDoc

Why should you use AsciiDoc?



QA|WARE

- Widespread support (e.g. IntelliJ and GitLab)
- It hits the **sweet-spot** between **simplicity** and **control**
 - Markdown
 - Too simple 🚀
 - multiple flavours 🍇
 - LaTeX
 - Too complex 🚀
 - infinite number of flavours ❄️
 - AsciiDoc
 - Just right 🍷
 - One standard 🍷



AsciiDoc killer features



QAWARE

Example 54. Source block with callouts

```
[source, ruby]
```

```
----
```

```
require 'sinat
```

```
get '/' do
```

```
  "Hello World!"
```

```
end
```

```
----
```

```
<1> L
```

```
<2> U
```

```
<3> H
```

▼ View

req

get

end

▼ Click

This

① L

② U

③ HTTP respons

Examp

.Cl

[%co

====

This

The re

▼ Click

This

Example 70. Table with three columns

```
Applications
```

```
[cols="1,1,2"] ①
```

```
|===
```

```
|Name|Category|Description|
```

```
|Firefox|Browser|
```

```
|Mozilla Firefox is a
```

```
|It's designed for s
```

```
|performance, portab
```

```
|===
```

```
|Arquillian
```

```
|Testing
```

```
|An innovative and
```

```
|Empowers developers
```

```
|===
```

```
① In this example, the co
```

```
sets their relative width
```

▼ View result of Example 70

Table 1. Applications

```
Name
```

```
Firefox
```

```
Arquillian
```

Example 76. Table with three columns

```
[cols="e,m,s"]
```

```
|===
```

```
|1> s|2> l|3> s
```

```
^|5 2.2+^|4
```

```
^|8
```

```
|9 2+>|10
```

```
|===
```

▼ View result of Example 76

```
1 2
```

```
5 6
```

```
8
```

```
9
```

Example 4. Block image macro with attribute list

```
.A mountain sunset ①
```

```
[#img-sunset, link=https://www.flickr.com/photos/javh/5448336655] ②
```

```
image::sunset.jpg[Sunset,200,100] ③ ④
```

- ① Defines the title of the block image, which gets displayed underneath the image when rendered.
- ② Assigns an ID to the block and makes the image a link. The `link` attribute can also be defined inside the attribute list of the block macro.
- ③ The first positional attribute, `Sunset`, is the image's alt text.
- ④ The second and third positional attributes define the width and height, respectively.

The result of Example 4 is displayed below.



Figure 1. A mountain sunset

AsciiDoc killer features



QA|WARE

- **Modularity** — Works well with multiple files
- **TOC** — Table of contents is a built-in feature
- **References** — Titles and explicit anchors can be used as ref-targets
- **Tables** — Easier to build than in Markdown, but still more powerful
- **Examples** — Easily show code examples (and explain it with ...)
- **Code Callouts** — Explaining code is very easy
- **Collapsibles** — Built-in feature to expand content
- **Element Titles** — Images, examples, etc. can have titles

ReadTheDocs unterstützt nur Markdown.



QA|WARE

Leider unterstützt ReadTheDocs momentan nur Markdown, weshalb ihr für eure Projektarbeit eure Dokumentation in Markdown schreiben solltet. ⚠